



## UNIVERSIDAD DE SANTIAGO DE CHILE

**Facultad de Ingeniería**

**Departamento de Ingeniería Mecánica**


---

# Laboratorio Fundamentos de Robótica Exp. 1: Construcción y desarrollo de un manipulador robótico.

---


**Nombre de los estudiantes:**
Cristóbal Calixto Jorquera; Rafael Palma Arenas; Matías Vásquez Pino.

**Profesores de teoría:** Arturo Álvarez Cea.

**Profesor de laboratorio:** Diego Luna Segura.



Santiago, Chile
2026

# Índice

**1. Resumen** **3**

**2. Objetivos** **3**
2.1. Objetivos generales . . . 3
2.2. Objetivos específicos . . . 3

**3. Introducción** **4**
3.1. Presentación del problema . . . 4

**4. Características técnicas del equipo** **5**

**5. Solución del problema** **6**
5.1. Integración de hardware y software . . . 9

**6. Presentación de resultados** **10**
6.1. Matriz de transformación homogénea . . . 10
6.2. Aplicación móvil y configuración inalámbrica . . . 11
6.3. Simulación del firmware . . . 11
6.4. Pruebas en el brazo físico . . . 12

**7. Conclusiones** **13**
7.1. Teóricas . . . 13
7.2. Aporte personal . . . 13

**8. Apéndices** **13**
8.1. Apéndice A: **Marco Teórico** . . . 13
8.2. Apéndice B: **Ecuaciones** . . . 15

**9. Anexos** **16**
9.1. Rutina de código . . . 16
9.2. Desarrollo algoritmo D-H . . . 18
9.3. Esquemático eléctrico . . . 19
9.4. Repositorio oficial de GitHub . . . 19

**10. Bibliografía** **19**

# Índice de figuras

1. Controlador ESP32 DevKit . . . 5
2. Driver PCA-9685 . . . 5
3. Sensor VL53L0X . . . 5
4. Motores MG90S . . . 5
5. Motores SG90 . . . 5
6. Cableado principal . . . 6
7. Cuerpo Impreso 3D . . . 6
8. Diagrama conexiones eléctricas . . . 8
9. Arquitectura de hardware y software . . . 9
10. Interfaz pantalla de inicio. . . . 11
11. Interfaz modo manual . . . 11
12. Ángulos de las articulaciones durante la rutina simulada con 3 cubos . . . 12
13. Diagrama de cada pieza del componente . . . 19

# Índice de cuadros

1. Matriz $A_{0->1}$ . . . 6
2. Matriz $A_{1->2}$ . . . 7
3. Matriz $A_{2->3}$ . . . 8
4. Matriz $A_{3->4}$ . . . 8
5. Cambio a unidades algebraicas. . . . 10
6. Matriz transformada homogénea, algebraica, con $r = a_1 + L_2c_2 + L_3c_{23} + L_4c_{234}$ 10
7. Matriz transformada homogénea en la pose de agarre . . . 10
8. Resultados de la simulación del firmware . . . 11
9. Parámetros D-H sistema . . . 18

# 1. Resumen

En el presente informe, se detallarán la construcción e implementación de un manipulador robótico, de 4 Grados de libertad.
Se incluye el proceso de construcción del dispositivo físico, componentes utilizados y diagramas eléctricos; así como también la determinación del movimiento de los motores, la verificación de los sensores y la construcción del algoritmo de Denavit-Hartenberg.
El firmware se validó en simulación, compilando el código real sobre un hardware emulado: se aprobaron las 34 pruebas de lógica, incluida la detección y el apilado de 3 cubos. En el brazo físico, en cambio, los problemas de agarre entre servos y eslabones impidieron ejecutar la rutina completa, por lo que ese objetivo no se cumplió en el prototipo.

# 2. Objetivos

## 2.1. Objetivos generales

Implementar un robot manipulador de laboratorio a escala impreso en 3D, que sea capaz de trasladar objetos y lograr ordenarlos, mediante el uso de servomecanismos con un controlador open hardware.

## 2.2. Objetivos específicos

- Construir manipulador robótico fabricado en base a impresión 3D, unir eslabones y fijaciones mediante pernos y tornillos.
- Integrar hardware a estructura del manipulador, es decir: acoplamiento de actuadores (servomotores), integración de hardware de control (Arduino UNO / ESP32), integración de hardware de comunicación (si es necesario).
- Implementar control manual y automático desde aplicación móvil Android.
- Integrar sensores en el sistema de manipulación para detectar de manera confiable la presencia y ubicación de los objetos a trasladar, asegurando la correcta activación de las rutinas de agarre, movimiento y depósito.

# 3. Introducción

## 3.1. Presentación del problema

Se debe implementar un brazo robótico, que sea capaz de posicionarse utilizando el modelo matemático de Denavit–Hartenberg, (O también conocido como D-H), de modo que sea capaz de ubicar su elemento terminal, (pinza), en distintas posiciones para trasladar objetos desde un punto inicial hasta dejarlos apilados en un punto final.
Se busca resolver los siguientes desafíos:

- Emular el traslado de objetos de dimensiones y peso conocidos desde puntos específicos del área de trabajo.
- Detectar la existencia de elementos a manipular en dichos puntos conocidos mediante el sensor correspondiente.
- Desarrollar una rutina de movimiento que traslade los bloques desde el punto detectado hasta el punto final, de modo que queden apilados o correctamente ordenados.
- Incorporar una aplicación móvil que permita interactuar con el manipulador y el controlador.
- Visualizar las posiciones determinadas de las articulaciones del manipulador mediante el algoritmo de D-H.
- Ejecutar rutinas automáticas de movimiento para trasladar objetos cúbicos conocidos y soportables por el manipulador.

# 4. Características técnicas del equipo

Para la construcción del sistema físico, se han recurrido a los siguientes elementos:

1. Controlador principal: **ESP32 DevKit (ESP32-WROOM-32)**
   Cerebro principal del funcionamiento del manipulador, tiene el código implementado, se encarga de recibir los datos del sensor y los computa mediante el algoritmo, para enviar la respuesta que deben realizar los motores.



Figura 1: Controlador ESP32 DevKit

2. Driver de potencia: **PCA-9685**
   Chip de expansión que permite al controlador manipular hasta 16 servo utilizando solo 2 pines del controlador, lo que reduce en gran medida la cantidad de cables y configuración del dispositivo.



Figura 2: Driver PCA-9685

3. Sensor distancia: **VL53L0X**
   El principal dispositivo que entrega información espacial al controlador, ubicado cerca de la pinza, recolecta la información de distancia frente al objeto, para detectar si hay un elemento en cada punto conocido.



Figura 3: Sensor VL53L0X

4. Motores principales:

- **Servomotores MG90S**
  Estos motores fueron los elegidos para las tres primeras articulaciones del manipulador, ubicados en la **base, hombro y codo**. Estos servo se caracterizan por ser más fuertes que los servo SG90, pero igual de ligeros, lo que evita que se aplique más esfuerzo del necesario en cada articulación.



Figura 4: Motores MG90S

- **Servomotores SG90**
  Planteados inicialmente para la **muñeca y las pinzas** del robot, son ideales para realizar movimientos que no requieran mucho esfuerzo de forma precisa. En los últimos desarrollos se reemplazaron por MG90S, ya que uno se dañó y estos no tienen engranajes metálicos.



Figura 5: Motores SG90

5. Cableado principal: **Cables Dupont**
   Cables que permiten conectarse fácilmente entre ellos, además de tener distintos formatos, siendo los utilizados para el manipulador los tipo Macho-Hembra.




Figura 6: Cableado principal

6. Cuerpo Principal: **Impresión 3D**
   El cuerpo del manipulador fue impreso enteramente en impresión 3D, ya que es una opción accesible, muy ligero y sobretodo económico para prototipar un proyecto de esta magnitud.

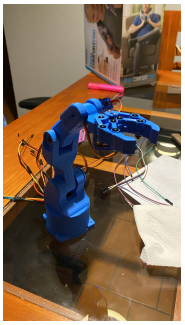


Figura 7: Cuerpo Impreso 3D

## 5. Solución del problema

Para lograr la cinemática directa del manipulador, se usará el algoritmo de Denavit-Hartenberg, que utiliza matrices para orientar la pinza del robot y determinar el movimiento de los motores. Para ello, el robot necesita los valores de los ángulos de movimiento, que se obtienen con la cinemática inversa de puntos conocidos de la mesa (Apéndice B), y los valores del largo de las piezas que conforman al manipulador. El sensor de distancia no entrega los ángulos: se usa para detectar si hay un elemento en cada punto.

$$L_1 = 0,032\ [m]\ L_2 = 0,067\ [m]\ L_3 = 0,0604\ [m]\ L_4 = 0,085\ [m]\ a_1 = -0,0012\ [m] \quad (1)$$

Donde $a_1$ es el desplazamiento horizontal del hombro respecto del eje de la base. Además, el sistema $\{0\}$ está a $H_{base} = 0,035\ [m]$ sobre la mesa.
Y las matrices de transformación básica para cada eslabón

| | | | |
|---|---|---|---|
| $Cos(q_1)$ | 0 | $Sen(q_1)$ | $a_1 * Cos(q_1)$ |
| $Sen(q_1)$ | 0 | $-Cos(q_1)$ | $a_1 * Sen(q_1)$ |
| 0 | 1 | 0 | $L_1$ |
| 0 | 0 | 0 | 1 |

Cuadro 1: Matriz $A_{0->1}$

| | | | |
|---|---|---|---|
| $Cos(q_2)$ | $-Sen(q_2)$ | 0 | $L_2 * Cos(q_2)$ |
| $Sen(q_2)$ | $Cos(q_2)$ | 0 | $L_2 * Sen(q_2)$ |
| 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 1 |

Cuadro 2: Matriz $A_{1->2}$

$$\begin{vmatrix} Cos(q_3) & -Sen(q_3) & 0 & L_3 * Cos(q_3) \\ Sen(q_3) & Cos(q_3) & 0 & L_3 * Sen(q_3) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Cuadro 3: Matriz $A_{2->3}$

$$\begin{vmatrix} Cos(q_4) & -Sen(q_4) & 0 & L_4 * Cos(q_4) \\ Sen(q_4) & Cos(q_4) & 0 & L_4 * Sen(q_4) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Cuadro 4: Matriz $A_{3->4}$

Para el sistema físico, se planteó la siguiente configuración eléctrica. En el diagrama, la muñeca y la pinza aparecen con los servos SG90 del diseño inicial; en los últimos desarrollos se usan MG90S, con las mismas conexiones:

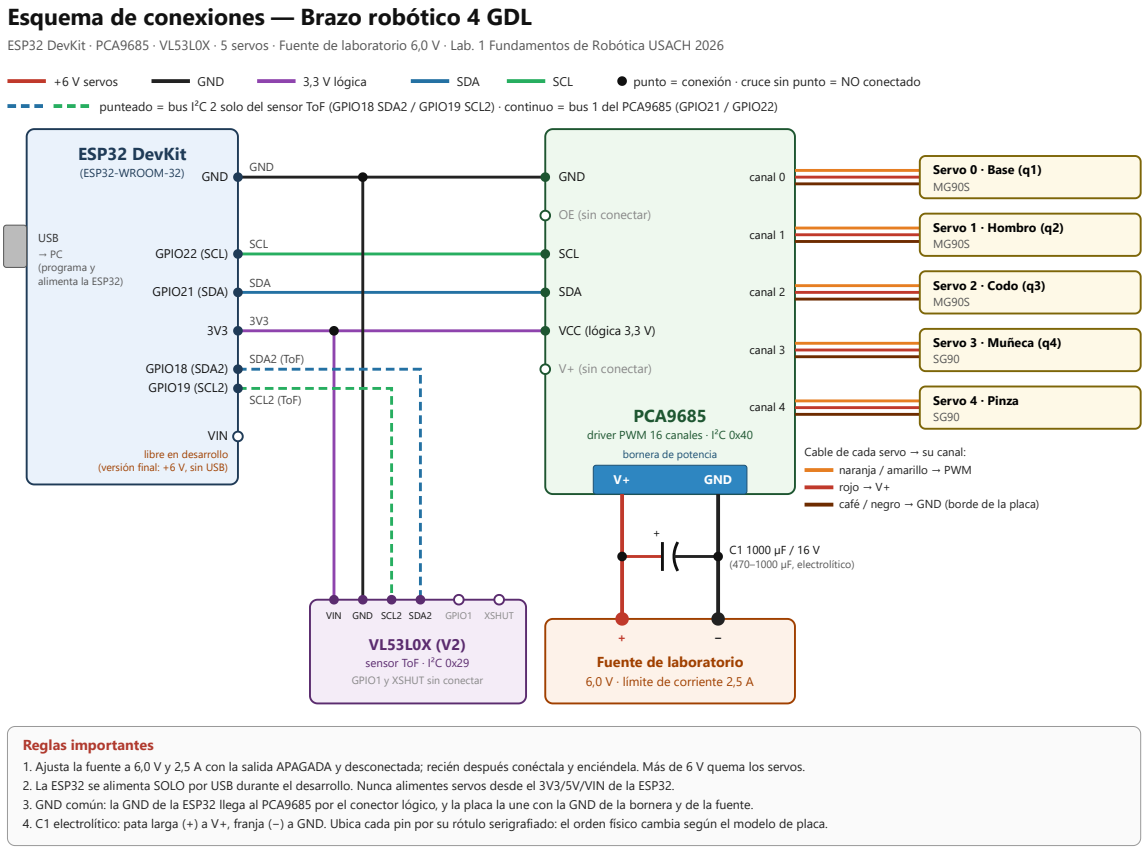



Figura 8: Diagrama conexiones eléctricas

## 5.1. Integración de hardware y software

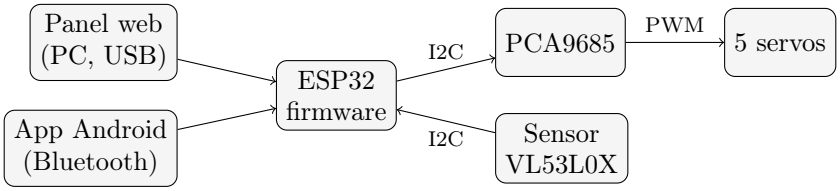



Figura 9: Arquitectura de hardware y software

El firmware recibe comandos de texto por USB o Bluetooth, los valida y mueve los servos. Sus partes principales son:

- **Calibración:** con dos poses medidas (A y B) y dos límites, cada servo obtiene la recta pulso–ángulo D-H, que se guarda en la memoria de la ESP32.
- **Trayectorias:** entre dos poses, las articulaciones siguen un perfil quíntico y llegan a la vez; para subir y bajar, la cinemática inversa se resuelve en cada paso para que la punta vaya en línea recta.
- **Telemetría D-H:** una tarea paralela calcula las matrices con los ángulos actuales y envía la posición de cada articulación, que el panel dibuja en vivo.
- **Rutina automática:** en cada punto de recogida, el sensor mide la distancia y la compara con la de la mesa vacía. Si detecta un cubo, lo toma, lo traslada a una altura segura y lo apila; si no quedan cubos, vuelve a la posición inicial.
- **Seguridad:** todo objetivo se limita al rango calibrado; las órdenes de parar y soltar se atienden incluso durante un movimiento; un servo quieto en su límite retrocede y, tras 90 s sin órdenes, se sueltan todos.

# 6. Presentación de resultados

## 6.1. Matriz de transformación homogénea

Una vez que se lleva a cabo la multiplicación de las 4 matrices básicas, se llega al siguiente resultado, considerando el siguiente cambio algebraico:

| | | | |
|---|---|---|---|
| $Cos(q_1) =$ | $c_1$ | $Sen(q_1) =$ | $s_1$ |
| $Cos(q_2) =$ | $c_2$ | $Sen(q_2) =$ | $s_2$ |
| $Cos(q_2 + q_3) =$ | $c_{23}$ | $Sen(q_2 + q_3) =$ | $s_{23}$ |
| $Cos(q_2 + q_3 + q_4) =$ | $c_{234}$ | $Sen(q_2 + q_3 + q_4) =$ | $s_{234}$ |

Cuadro 5: Cambio a unidades algebraicas.

$$\begin{vmatrix} c_1c_{234} & -c_1s_{234} & s_1 & c_1\,r \\ s_1c_{234} & -s_1s_{234} & -c_1 & s_1\,r \\ s_{234} & c_{234} & 0 & L_1 + L_2s_2 + L_3s_{23} + L_4s_{234} \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Cuadro 6: Matriz transformada homogénea, algebraica, con $r = a_1 + L_2c_2 + L_3c_{23} + L_4c_{234}$

Como comprobación numérica, en la pose de agarre $q = (0°;\ 49,39°;\ -76,86°;\ -62,53°)$, en milímetros:

$$\begin{vmatrix} 0 & 1 & 0 & 96{,}0 \\ 0 & 0 & -1 & 0 \\ -1 & 0 & 0 & -30{,}0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Cuadro 7: Matriz transformada homogénea en la pose de agarre

La punta queda a 96 mm de la base y, sumando $H_{base} = 35$ mm, a 5 mm sobre la mesa, que es la altura de agarre pedida a la cinemática inversa.

## 6.2. Aplicación móvil y configuración inalámbrica

Para la implementación de la aplicación para las subrutinas del manipulador, se utilizó el software de *MIT App Inventor*, mediante diseño y programación en bloques.

La aplicación se encuentra disponible para dispositivos android y el instalador de esta puede encontrarse en el github anexado al final de este informe.




Figura 10: Interfaz pantalla de inicio.

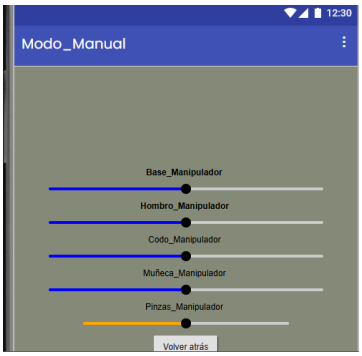



Figura 11: Interfaz modo manual

## 6.3. Simulación del firmware

Como las pruebas en el brazo físico quedaron limitadas por problemas mecánicos, la lógica se validó en simulación. El código real del firmware se compiló en el computador sobre un hardware emulado: un PCA9685 que guarda sus registros como el chip real, un sensor que calcula la distancia según la pose del brazo y los cubos sobre la mesa, y la memoria de la ESP32. Se le envían los mismos comandos que recibiría por USB y se revisa lo que llega a cada servo.
El mundo simulado es ideal, sin holguras, peso ni deslizamiento. Por eso, estas pruebas validan la lógica del programa, pero no el comportamiento mecánico del brazo.

| Prueba | Resultado |
|---|---|
| Pruebas de lógica aprobadas | 34 de 34 |
| Calibración y poses de la tarea alcanzables | Correcto |
| Rutina sin cubos | 3 puntos vacíos; vuelve a la posición inicial |
| Cubo corrido 30 mm de su punto | No lo detecta; vuelve a la posición inicial |
| Rutina con 3 cubos | Apilados a 0, 20 y 40 mm, en 57,2 s |
| Altura mínima de la punta sobre la mesa | 4,6 mm |
| Parar y soltar durante la rutina | Se detiene; soltar corta los pulsos |
| Pose fuera de límites o de alcance | Rechazada; el brazo no se mueve |
| Firmware (C++) frente a modelo independiente (Python) | Diferencia máxima de 0,001 mm en 1145 muestras |

Cuadro 8: Resultados de la simulación del firmware

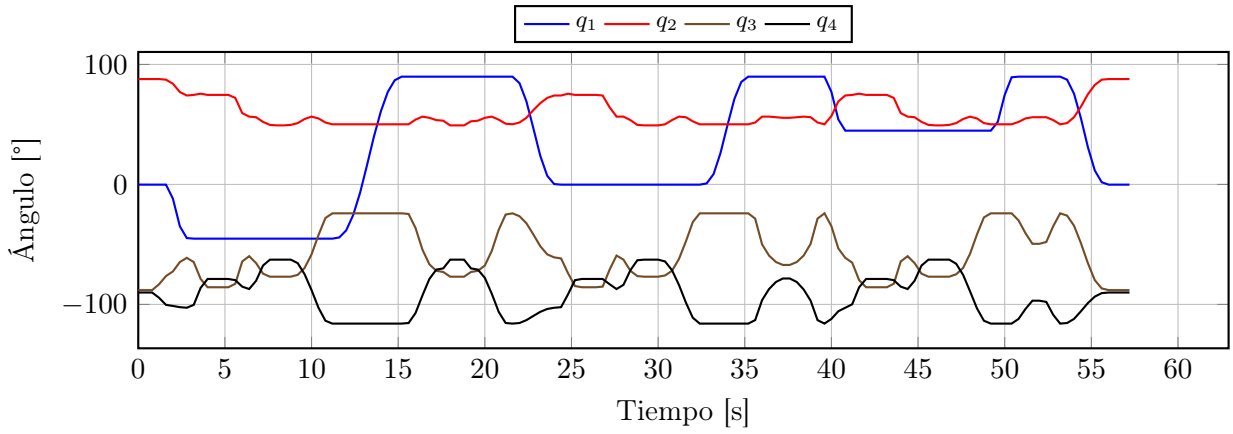



Figura 12: Ángulos de las articulaciones durante la rutina simulada con 3 cubos

### 6.4. Pruebas en el brazo físico

Con el brazo conectado, la comunicación y la electrónica respondieron bien: se detectaron el PCA9685 y el sensor, sin errores en el bus I2C, y el firmware envió el brazo a la pose inicial sin errores. Durante las pruebas también se detectó que abrir y cerrar la pinza estaban invertidos por una mala calibración, lo que se corrigió.

Sin embargo, el brazo no ejecuta correctamente los movimientos. Presenta problemas de agarre en puntos clave de los eslabones, donde el servo se une a la pieza; por eso, muchas veces los servos se sobrecalientan o no ejecutan los movimientos necesarios para la rutina. En particular, el codo llegó a quedar forzado y bloqueado contra la estructura. Por ello, la rutina automática no se pudo completar en el brazo físico y no se cumple lo esperado en esa parte. Los servos MG90S no informan su posición ni su corriente, de modo que el firmware no puede detectar estas fallas por sí mismo.

# 7. Conclusiones

## 7.1. Teóricas

En este informe, se ha presentado el proceso de desarrollo de un manipulador cinemático de 4 Grados de Libertad, mediante cálculos matriciales y configuraciones inalámbricas.
Se han utilizado herramientas de programación como C++ para el firmware, programación en bloques para la aplicación y Python para la verificación del modelo.
Los archivos de impresión fueron adquiridos mediante la plataforma Cults3D, enlace en la bibliografía. El análisis cinemático fue realizado mediante el algoritmo de Denavit-Hartenberg, con la Matriz Transformada Homogénea. El análisis cinemático inverso se realizó mediante trigonometría y ecuaciones pitagóricas.
La simulación mostró que la lógica del firmware cumple la secuencia pedida: detecta, traslada y apila 3 cubos dentro de los límites definidos. La principal limitación está en la parte mecánica: el agarre entre servos y eslabones no permitió llevar esa lógica al brazo físico.

## 7.2. Aporte personal

El *Dandori*, concepto proveniente del japonés, hace alusión al arte de la **planificación, organización y preparación** antes de llevar a cabo un proyecto.
Como equipo, llevar a cabo este proyecto nos permitió familiarizarnos con el *Dandori*, y prepararnos para los desafíos que los próximos laboratorios pondrán sobre la mesa.
Entendimos de mejor manera los pasos para la confección de los sistemas, tanto físicos como digitales, y se espera con ansias observar la evolución del equipo de trabajo en el desarrollo de los siguientes proyectos.

# 8. Apéndices

## 8.1. Apéndice A: Marco Teórico

La palabra robot se define como una maquina controlada por un ordenador y programada para moverse, manipular objetos y realizar trabajos a la vez que interacciona con su entorno. Los robots son capaces de realizar tareas repetitivas de forma mas rápida, barata y precisa que los seres humanos.
El término procede de la palabra checa *robota*, que significa 'trabajo obligatorio'. Desde entonces se ha empleado la palabra robot para referirse a una máquina que realiza trabajos para ayudar a las personas para efectuar tareas difíciles o de riesgo para los humanos. Mientras tanto la robótica se define como la rama de la inteligencia artificial que estudia los sistemas automáticos capaces de establecer una interacción directa con el mundo físico.*(Pattenden, 2014)*

Los robots pueden ser clasificados como: fijos o industriales y móviles:

- El robot fijo se utiliza para llevar a cabo tareas que pueden ser peligrosas para el hombre, o simplemente para mejorar la producción en la industria realizando tareas repetitivas.
- Por otro lado, el robot móvil, tiene la capacidad de desplazarse de un lugar a otro con ayuda

de un sistema de locomoción, siguiendo su camino de acuerdo a la información recibida a través de un sistema de sensores.*(Pattenden, 2014)*.

## 8.2. Apéndice B: Ecuaciones

Para los cálculos de las proyecciones de los ángulos necesarios para completar la matriz transformada homogénea, se utilizan las coordenadas del punto al que debe llegar la pinza, además de las siguientes ecuaciones:

- El ángulo de la base se calcula como: $q_1 = \arctan \frac{Y_0}{X_0}$.
- Con la observación anterior, se obtiene el módulo como: $\sqrt{X_0^2 + Y_0^2}$.
- Además se necesitan otros parámetros:
  $L_{x4} = L_4 * \cos q$ ; $L_{z4} = L_4 * \sin q$

  $X_1 = Modulo - a_1 - L_{x4}$ y $Z_1 = Z_0 - L_{z4} - L_1$, con $Z_0$ medido desde el sistema $\{0\}$.

  $h = \sqrt{X_1^2 + Z_1^2}$.

  Siendo $q$, entregado por el usuario.
- Con los parámetros obtenidos, se definen las ecuaciones restantes:

  $\alpha_1 = \arctan \frac{Z_1}{X_1}$

  $\alpha_2 = \arccos \frac{L_2^2 + h^2 - L_3^2}{2hL_2}$

  $q_2 = \alpha_1 + \alpha_2$
  $\alpha_3 = \arccos \frac{L_2^2 + L_3^2 - h^2}{2L_2L_3}$

  $q_3 = \alpha_3 - 180$
  $q_4 = q - q_2 - q_3$
- Estos valores, al ser calculados, completan la matriz D-H para el movimiento del manipulador.

# 9. Anexos

## 9.1. Rutina de código

Para este proyecto, se desarrollaron 2 códigos, uno en el controlador ESP-32 y una aplicación móvil.

- ESP-32: El microcontrolador utiliza el driver de potencia para alimentar y controlar 5 servos de forma secuencial, mediante la **Matriz Transformada Homogénea**, y haciendo uso del hardware PCA-9685, placa que alimenta los servomotores por separado y evita interferencias en las señales de transmisión.
  El código del microcontrolador utiliza las instrucciones del usuario para iniciar las subrutinas.
  En el modo manual, el microcontrolador recibe un comando de texto único para cada servo, en conjunto con la cantidad de grados que el usuario quiera rotarlo.
  En el modo autónomo, el usuario solo puede encender o apagar el ciclo de detección y manipulación de los elementos dispuestos para la demostración.

- Aplicación móvil: El algoritmo de la aplicación es simple, se inicializa la aplicación con 4 botones básicos, 2 que inician subrutinas, 1 para salir y 1 para conectar el bluetooth de la placa ESP-32 al teléfono.
  El primer modo, automático, le informa al manipulador que inicie la secuencia automática de detección y manipulación de objetos en su entorno, pudiendo detenerse en cualquier momento mediante un switch.
  El segundo modo, manual, permite al usuario manipular 4 slidders de 0 a 180 grados, para manejar las 4 articulaciones de movimiento del manipulador, además de un quinto slidder más pequeño que permite abrir y cerrar las pinzas del manipulador.

El firmware, las simulaciones y el instalador de la aplicación se encuentran anexados en el repositorio Github ubicado al final de esta sección. A continuación se muestran las dos partes centrales del firmware.

**Cinemática inversa:**

```
inline bool ik(const Pose &p, float q[4]) {
  float phi = radians(p.phi);
  float r = hypotf(p.x, p.y);
  // Paso 1: se resta el eslabon L4 para obtener el centro de la muneca,
  // en el plano vertical del brazo (rw = distancia horizontal, zw = altura desde el hombro).
  float rw = r - A1_DH - L4 * cosf(phi);
  float zw = (p.z - H_BASE) - L1 - L4 * sinf(phi);
  // Paso 2: ley del coseno en el triangulo hombro-codo-muneca. D = cos(q3).
  // Si |D| > 1 el triangulo no existe: el punto esta fuera de alcance.
  float D = (rw * rw + zw * zw - L2 * L2 - L3 * L3) / (2.0f * L2 * L3);
  if (fabsf(D) > 1.0f) return false;
  // Seno negativo: se elige la solucion de codo arriba.
  float s3 = -sqrtf(1.0f - D * D);
  float q3 = atan2f(s3, D);
  // Paso 3: angulo hacia la muneca menos el angulo interior del triangulo.
  float q2 = atan2f(zw, rw) - atan2f(L3 * s3, L2 + L3 * D);
  // Paso 4: la base apunta hacia el punto y q4 completa la inclinacion pedida.
  q[0] = degrees(atan2f(p.y, p.x));
```

```
  q[1] = degrees(q2);
  q[2] = degrees(q3);
  q[3] = p.phi - q[1] - q[2];
  return true;
}
```

**Rutina automática con sensor:**

```
void rutinaAutomatica() {
  if (!exigirCalibracion(0x1F)) return;
  if (!tofOk) {
    responder("ERR,sensor␣ToF␣no␣disponible\n");
    return;
  }
  if (!tofAprendido) {
    responder("ERR,primero␣ejecuta␣'aprender'␣con␣los␣puntos␣de␣recogida␣vacios\n");
    return;
  }
  responder("E,INICIO\n");
  int nivel = 0;
  bool ok = moverPinza(PINZA_ABIERTA);
  bool hallado = true;
  // Recorre los puntos de recogida hasta que una pasada completa no encuentra cubos o la pila se llena.
  while (ok && modoAuto && hallado && nivel < MAX_PILA) {
    hallado = false;
    for (int i = 0; ok && modoAuto && i < N_RECOGIDA && nivel < MAX_PILA; i++) {
      responder("E,BUSCANDO,%d\n", i + 1);
      ok = irSeguro(poseObservacion(PUNTOS_RECOGIDA[i]));
      if (!ok) break;
      delay(150);
      uint16_t d = medirTof(false);
      bool hay = d + CUBO / 2 < baseTof[i];
      responder("E,DISTANCIA,%d,%u,%u,%s\n", i + 1, d, baseTof[i], hay ? "CUBO" : "VACIO");
      if (!hay) continue;
      hallado = true;
      responder("E,TOMANDO,%d\n", i + 1);
      ok = tomar(i);
      if (!ok) break;
      responder("E,APILANDO,%d\n", nivel + 1);
      ok = dejar(nivel);
      if (ok) nivel++;
    }
  }
  if (pararPedido) {
    responder("E,PARADO\n"); // queda detenido donde esta
  } else {
    responder(ok ? "E,FIN,%d\n" : "E,ERROR,%d\n", nivel);
    irHome();
  }
  modoAuto = false;
}
```

## 9.2. Desarrollo algoritmo D-H

A continuación se especificarán los pasos del algoritmo D-H, y el resultado implementado en el manipulador:

1. Numerar los eslabones: Empezando con la base como 0.

2. Numerar y localizar las articulaciones: Cada intersección de los eslabones en una articulación, se debe empezar desde el 1. Luego se define su eje $Z$, si es una articulación rotatoria, será su eje de giro, si es prismática, será su eje de desplazamiento.

3. Establecer el sistema de la base: Sobre el eje $Z_o$ se deben incluir los ejes $X_0$ y $Y_0$, siempre que cumplan como sistema **dextrógiro** con la regla de la mano derecha.

4. Localizar los ejes del resto de las articulaciones: Para cada articulación $Z_{i+1}$, se deben definir sobre el eje de movimiento previamente definido en el paso 2. Luego considera si la articulación anterior tiene su eje $Z_i$ cortando el eje $Z_{i+1}$, si es el caso, se establece ese punto como nuevo origen para $X_{i+1}$,apuntando hacia el eje $Z_i$ y $Y_{i+1}$; caso contrario, si ambos están paralelos, se elijen $X_{i+1}$ y $Y_{i+1}$ a conveniencia, haciendo coincidir orígenes o alineándolos; manteniendo la regla de la mano derecha.

5. Se llena la tabla de parámetros D-H: Son 4.

   - $\theta_i$ (ángulo de articulación): Ángulo de rotación alrededor del eje $z_{i-1}$ para alinear $X_{i-1}$ con $X_i$.
   - $d_i$ (desplazamiento de articulación): Distancia a lo largo del eje $z_{i-1}$ desde el origen $O_{i-1}$ hasta la intersección con $X_i$.
   - $\alpha_i$ (torsión del eslabón): Ángulo de rotación alrededor del eje $X_i$ para alinear $z_{i-1}$ con $z_i$.
   - $a_i$ (longitud del eslabón): Distancia a lo largo del eje $X_i$ desde $z_{i-1}$ hasta $z_i$.

   La matriz de parámetro para este manipulador queda de la siguiente manera.

| Articulación | $\theta_i$ | $d_i$ | $\alpha_i$ | $a_i$ |
|---|---|---|---|---|
| I | $q_1$ | $L_1$ | $\pi/2$ | $a_1$ |
| II | $q_2$ | 0 | 0 | $L_2$ |
| III | $q_3$ | 0 | 0 | $L_3$ |
| IV | $q_4$ | 0 | 0 | $L_4$ |

Cuadro 9: Parámetros D-H sistema

6. Matrices de transformación homogénea: Multiplica las cuatro transformaciones básicas (rotación en Z, traslación en Z, traslación en X, rotación en X) para obtener la matriz $A_i$ de cada eslabón y encadenarlas en una matriz de transformación total T.
   Véase el resultado aquí

### 9.3. Esquemático eléctrico

Se presenta a continuación el esquemático eléctrico de los equipos físicos.

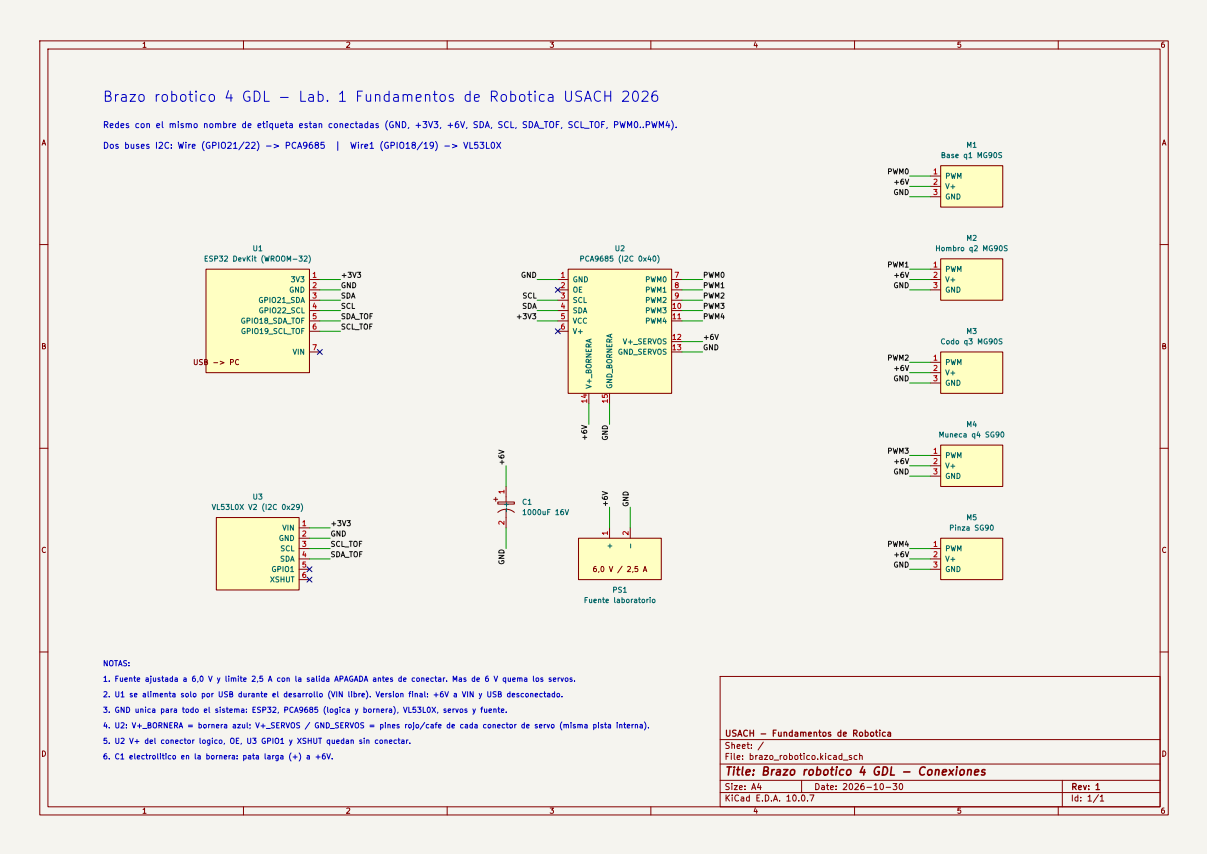



Figura 13: Diagrama de cada pieza del componente

### 9.4. Repositorio oficial de GitHub

Los códigos, los anexos, los archivos, la aplicación y los resultados se encuentran disponibles en el siguiente repositorio de GitHub:

**Repositorio de archivos completos en GitHub.**

## 10. Bibliografía


- Diseño y archivos STL originales. https://cults3d.com/:2382816
- Pattenden, O. (2014). Marco teórico del brazo robótico [Documento adjunto]. Scribd. https://es.scribd.com/document/238035168/Marco-Teorico-Del-Brazo-Robotico
- Sanz Benito, R. (2018, 24 de septiembre). Análisis cinemático de un brazo robótico de 4GDL con lenguaje Python [Diapositivas]. SlideShare. https://es.slideshare.net/slideshow/anlisis-cinemtico-de-un-brazo-robtico-de-4gdl-con-lenguaje-python/116240217

- How To Mechatronics. DIY Arduino Robot Arm with Smartphone Control. Proyecto de ejemplo indicado en el enunciado.
  https://howtomechatronics.com/tutorials/arduino/diy-arduino-robot-arm-with-smartphone-control/